

Guide Débutant : GitHub avec VS Code

Un pas-à-pas concis

26 octobre 2025

Table des matières

1	Ce que vous allez apprendre	1
2	Prérequis	2
3	Choisir votre mode d'authentification	2
3.1	Option A : HTTPS + Personal Access Token	2
3.2	Option B : Clés SSH (recommandé sur le long terme)	2
4	Deux flux de travail courants	3
4.1	A. Démarrer un nouveau projet local puis le publier sur GitHub	3
4.2	B. Cloner un dépôt GitHub existant	3
5	Les bases au quotidien dans VS Code	4
5.1	Modifier et valider (commit)	4
5.2	Pousser (push) vos commits	4
5.3	Récupérer (pull) les mises à jour	4
5.4	Voir l'historique et les différences	4
6	Branches et Pull Requests	5
6.1	Créer et changer de branche	5
6.2	Ouvrir une Pull Request (PR)	5
7	Gérer les conflits de fusion (aperçu rapide)	5
8	Ajouter .gitignore et un README	5
9	Dépannage	6
10	Mémo des commandes	7
11	Glossaire	7
12	Docs officielles utiles	7

1 Ce que vous allez apprendre

Ce court guide vous apprend l'essentiel pour utiliser **Git** et **GitHub** depuis **Visual Studio Code (VS Code)** :

— Créer un dépôt (repository) GitHub et le connecter à votre ordinateur.

- Modifier des fichiers, *valider* (commit), *pousser* (push = envoyer) et *recupérer* (pull = télécharger les mises à jour).
 - Utiliser des branches, ouvrir des Pull Requests (PR) et résoudre des conflits simples.
- Aucune connaissance préalable de Git n'est requise.

2 Prérequis

1. Un compte GitHub gratuit : github.com.
2. Installer Git : git-scm.com/downloads.
3. Installer VS Code : code.visualstudio.com.
4. (Optionnel, recommandé) Installer l'extension *GitHub Pull Requests and Issues* dans VS Code.

Configuration Git initiale (une seule fois)

Ouvrez un terminal (Terminal → New Terminal dans VS Code) et configurez votre identité :

```
# Utilisez votre vrai nom et le même e-mail que sur GitHub
git config --global user.name "Votre Nom"
git config --global user.email "votre.email@example.com"

# Utiliser 'main' comme branche par défaut pour les nouveaux dépôts
git config --global init.defaultBranch main

# (Optionnel) Toujours rebaser lors des pulls pour un historique linéaire
git config --global pull.rebase true
```

3 Choisir votre mode d'authentification

GitHub n'accepte plus les mots de passe pour Git via HTTPS. Utilisez **HTTPS avec un Personal Access Token (PAT)** ou des **clés SSH**. Les deux fonctionnent mais je vous conseille vivement d'utiliser **HTTPS**, c'est le plus simple si vous n'avez pas l'habitude des clés SSH.

3.1 Option A : HTTPS + Personal Access Token

1. Dans GitHub:Settings → Developer settings → Personal access tokens. Créez un token *fine-grained* ou *classic* avec le scope repo.
2. Quand Git demande un mot de passe lors d'un `git push`, collez le token (pas votre mot de passe). VS Code peut le mémoriser via Git Credential Manager.

3.2 Option B : Clés SSH (recommandé sur le long terme)

```
# Créer une nouvelle clé SSH (Entrée pour les valeurs par défaut)
ssh-keygen -t ed25519 -C "votre.email@example.com"

# Démarrer l'agent SSH et ajouter la clé privée
# macOS
eval "$(ssh-agent -s)" && ssh-add --apple-use-keychain ~/.ssh/id_ed25519
# Linux
eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519
# Windows (PowerShell, avec Git Bash)
ssh-agent.exe -s
```

```
ssh-add.exe ~/.ssh/id_ed25519

# Copier la clé publique et l'ajouter sur GitHub : Settings -> SSH and GPG keys
cat ~/.ssh/id_ed25519.pub

# Tester la connexion (tapez 'yes' la première fois)
ssh -T git@github.com
```

Sur GitHub, utilisez l'URL SSH (git@github.com:UTILISATEUR/DEPOT.git) pour cloner ou ajouter un *remote*.

4 Deux flux de travail courants

4.1 A. Démarrer un nouveau projet local puis le publier sur GitHub

Il est possible de tout faire depuis VSCODE mais je vous recommande d'utiliser le Terminal de votre ordinateur (chercher "Terminal" ou "Invite de commandes" dans le menu Démarrer sous Windows).

```
cd /chemin/vers/votre/projet #cette ligne est pour vous situer dans le bon dossier, "
    cd" signifie "change directory"

git init # Initialise un dépôt Git local
git add le_nom_du_fichier_ou_dossier
# ou : git add . pour tout ajouter (mais il vaut mieux être précis et ajouter chaque
    fichier)
# NE JAMAIS AJOUTER DES DONNEES SUR LE GIT CAR LES FICHIERS PEUVENT ETRE TROP LOURDS
git commit -m "Initial commit"
# "commit" signifie que vous validez les changements faits dans le projet
# -m permet d'ajouter un message pour expliquer ce que vous avez fait

# Créez un dépôt vide sur GitHub (via le site) et copiez son URL :
# HTTPS : https://github.com/UTILISATEUR/DEPOT.git
# ou SSH : git@github.com:UTILISATEUR/DEPOT.git

git remote add origin <VOTRE-URL-DEPOT> # lie le repos local à celui en ligne grâce à l'
    URL
git push -u origin main #pousse les changements locaux en ligne "origin" est le nom du
    dépôt distant, "main" est la branche principale (je pourrais vous expliquer plus
    en détails les branches mais pour l'instant retenez juste que c'est la branche
    principale)
```

4.2 B. Cloner un dépôt GitHub existant

Une fois que votre camarade a mis son projet en ligne, vous pouvez le cloner (copier) sur votre ordinateur pour y travailler. **Terminal (commandes) :**

```
# Avec l'URL HTTPS ou SSH copiée depuis GitHub
git clone <URL-DEPOT>
cd <dossier-depot>
code . # ouvrir dans VS Code
```

Si la commande code n'est pas reconnue, ouvrez VSCODE et faites CTRL+SHIFT+P puis tapez "Shell Command : Install 'code' command in PATH"

Si ça ne marche pas désinstaller la commande code en faisant CTRL+SHIFT+P puis tapez "Shell Command : Uninstall 'code' command in PATH"

et réinstallez la commande code en refaisant la première étape.

5 Les bases au quotidien dans VS Code

5.1 Modifier et valider (commit)

Terminal :

```
# Voir ce qui a changé
git status

# Indexer tout (modifié/supprimé/ajouté)
git add le_nom_du_fichier_ou_dossier
# ou : git add . pour tout ajouter (mais il vaut mieux être précis et ajouter chaque
      fichier)

# Valider avec un message explicite
git commit -m "Expliquez ce qui a changé"
```

5.2 Pousser (push) vos commits

Maintenant que vous avez des commits locaux, envoyez-les sur GitHub pour qu'ils apparaissent en ligne.

Terminal :

```
# Premier push sur une nouvelle branche :
git push -u origin main # ou : git push -u origin <votre-branche> (pour l'instant ne
      vous souciez pas des branches)

# Push suivants (après avoir défini l'upstream) :
git push
```

ATTENTION : Si vous travaillez à deux sur la même branche, il est préférable de faire un `git pull --rebase` avant chaque `git push` pour éviter les conflits. Sinon, vous risquez d'écraser le travail de votre camarade ou de devoir résoudre des conflits compliqués. S'om y a le moindre problème n'hésitez pas à me demander ce serait dommage de perdre du travail :).

5.3 Récupérer (pull) les mises à jour

Terminal :

```
# Récupérer les nouveaux commits du remote et rebaser vos changements
git pull --rebase
```

5.4 Voir l'historique et les différences

Pour voir l'historique des commits et les différences entre versions, vous pouvez utiliser :

- Vue **Timeline** (panneau de droite) pour l'historique par fichier.
- **Source Control** → cliquez sur un fichier pour un diff côte à côte.
- Palette de commandes (accessible en faisant `CTRL+SHIFT+P`) : **Git: View History**, ou l'extension **GitHub**.

6 Branches et Pull Requests

Cette partie n'est pas obligatoire pour débiter donc pour l'instant ne vous en souciez pas.

6.1 Créer et changer de branche

Pour collaborer plus facilement, vous pouvez créer une branche dédiée (par exemple **marie_branche**) qui vous permet de travailler seul sur une version du projet sans impacter la branche principale (**main**).

Terminal :

```
# Créer et basculer sur une nouvelle branche
git checkout -b marie_branche

# La pousser et définir l'upstream
git push -u origin marie_branche
```

6.2 Ouvrir une Pull Request (PR)

1. Poussez votre branche sur GitHub.
2. Sur GitHub, cliquez **Compare & pull request**.
3. Rédigez une description claire, puis **Create pull request**.
4. **Merge**. Supprimez la branche si souhaité.

7 Gérer les conflits de fusion (aperçu rapide)

Les conflits surviennent quand les mêmes lignes changent.

1. Un **pull** ou un **merge** déclenche le conflit. VS Code ouvre le **Merge Editor**.
2. Choisissez **Accept Current**, **Accept Incoming** ou **Accept Both**. Éditez à la main si nécessaire.
3. Testez en local. Puis **Stage** les fichiers résolus et **Commit**.
4. Si vous étiez en **rebase**, poursuivez avec **git rebase -continue**.

Un marqueur de conflit ressemble à ceci :

```
<<<<<<< HEAD
Vos modifications locales
=====
Leur modification depuis le remote
>>>>>>> origin/main
```

8 Ajouter .gitignore et un README

Exemples courants de .gitignore

C'est un détail mais peut-être utile à savoir pour plus tard.

```
# macOS / Windows
.DS_Store
Thumbs.db

# Éditeurs
.vscode/
```

```
.idea/  
  
# Node.js  
node_modules/  
dist/  
  
# Python  
__pycache__/  
*.pyc  
.venv/
```

README.md minimal

```
# Titre du projet  
  
Brève description de ce que fait ce projet et pour qui il est.  
  
## Comment exécuter  
Expliquez comment installer les dépendances et lancer l'application.  
  
## Liens utiles  
- Suivi des issues  
- Documentation
```

9 Dépannage

Échec d'authentification (HTTPS). Utilisez un Personal Access Token au lieu du mot de passe. Mettez à jour le remote si besoin :

```
# Remplacez par l'URL HTTPS ou SSH  
git remote set-url origin <NOUVELLE-URL>
```

Le remote origin existe déjà.

```
git remote -v  
# Si incorrect, corriger :  
git remote set-url origin <URL-CORRECTE>  
# Ou supprimer puis ré-ajouter :  
git remote remove origin  
git remote add origin <URL>
```

'fatal : The current branch has no upstream'.

```
# Le premier push doit définir l'upstream :  
git push -u origin <votre-branche>
```

Pull indique des historiques divergents ou propose un merge commit.

```
# Conserver un historique linéaire par rebase :  
git pull --rebase  
# Ou configurer une fois pour toutes :  
git config --global pull.rebase true
```

10 Mémo des commandes

<code>git status</code>	Voir ce qui a changé
<code>git add fichier</code>	Indexer les changements
<code>git commit -m "msg"</code>	Enregistrer les changements indexés avec un message
<code>git push</code>	Envoyer les commits vers GitHub
<code>git pull -rebase</code>	Récupérer les mises à jour et rebaser
<code>git checkout -b NOM</code>	Créer et basculer sur la branche NOM
<code>git switch NOM</code>	Basculer sur une branche existante NOM
<code>git log -oneline -graph</code>	Historique compact

11 Glossaire

Dépôt (repository) Projet suivi par Git, en local et/ou sur GitHub.

Commit Instantané de vos modifications avec un message.

Remote Copie de votre dépôt hébergée ailleurs (ex. GitHub). **origin** est le nom par défaut.

Branche Pointeur mobile vers une ligne de développement (ex. **main**, **feature/...**).

Push/Pull Envoyer / récupérer des commits vers / depuis un remote.

PR Pull Request : proposition de fusionner des changements d'une branche dans une autre, avec revue.

12 Docs officielles utiles

- Git : git-scm.com/docs
- GitHub : docs.github.com
- VS Code (Git) : [VS Code Source Control](https://code.visualstudio.com/docs/sourcecontrol)

Astuce : Faites de petits commits fréquents avec des messages clairs. Votre futur vous dira merci.